Spam Email Classifier (CLI Application)

Project Title: Spam Email Classifier (CLI Application)

Developed by: DHEERAJ KUMAR MAHTO

Registration Number: 25BCE10465

Course: Fundamentals in AI and Machine Learning (CSA2001)

INTRODUCTION

Email communication plays a vital role in modern digital interaction. However, the increase in email usage has also resulted in a rise in spam and phishing attacks. Spam emails often contain advertisements, fraudulent messages, or malicious links that may compromise user security.

Machine Learning techniques enable automatic classification of emails based on learned patterns from historical data. This project develops a Command Line Interface (CLI) application that classifies emails as **Spam** or **Ham (Legitimate)** using Natural Language Processing (NLP) and supervised learning algorithms.

MOTIVATION OF THE PROJECT

Manual spam identification is inefficient and prone to human error. Users often fall victim to phishing attacks due to lack of awareness.

The motivation of this project includes:

- Automating spam email detection
- Applying Artificial Intelligence concepts practically
- Understanding Machine Learning workflow
- Enhancing cybersecurity awareness

PROBLEM STATEMENT

Traditional rule-based filtering systems fail to detect modern spam and phishing emails because attackers continuously change message patterns.

The problem addressed in this project is:

To design an intelligent system capable of automatically identifying spam emails using Machine Learning techniques.

OBJECTIVE OF THE PROJECT

- To preprocess email text using NLP techniques
- To extract meaningful features from text data
- To train a Machine Learning classification model
- To evaluate model performance
- To develop an interactive CLI-based prediction system

EXISTING METHODS

Existing spam detection approaches include:

1. Rule-based filtering systems
2. Blacklist-based filtering
3. Keyword matching techniques
4. Manual moderation systems

PROS AND CONS OF THE STATED METHODS

Method	Advantages	Disadvantages
Rule-Based	Easy implementation	Easily bypassed
Blacklist	Fast filtering	Cannot detect new spam
Keyword Matching	Simple logic	High false positives

Manual Filtering	Accurate	Not scalable

Machine Learning overcomes these limitations by learning patterns automatically.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4GB
- Storage: 500MB free space

Software Requirements

- Python 3.8 or higher
- VS Code / Terminal
- Operating System: Windows / Mac / Linux

Libraries Used

- Pandas
- NumPy
- Scikit-learn
- NLTK

METHODOLOGY AND GOAL

The project follows a Machine Learning pipeline:

1. Dataset collection
2. Data preprocessing
3. Feature extraction using TF-IDF
4. Model training using Naive Bayes

5. Model evaluation
6. Email prediction through CLI

Goal: Develop an automated intelligent spam classification system.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

1. Data Module

Loads and validates dataset.

2. Preprocessing Module

- Text cleaning
- Lowercasing
- Stopword removal

3. Training Module

Trains Machine Learning model.

4. Evaluation Module

Calculates accuracy and performance metrics.

5. Prediction Module

Classifies user-input emails.

6. CLI Interface

Provides interactive user menu.

ALGORITHM DEVELOPMENT

Algorithm Used: Multinomial Naive Bayes

Steps:

1. Convert email text into TF-IDF vectors.
2. Calculate probability distribution of words.
3. Apply Bayes theorem.
4. Predict class with highest probability.

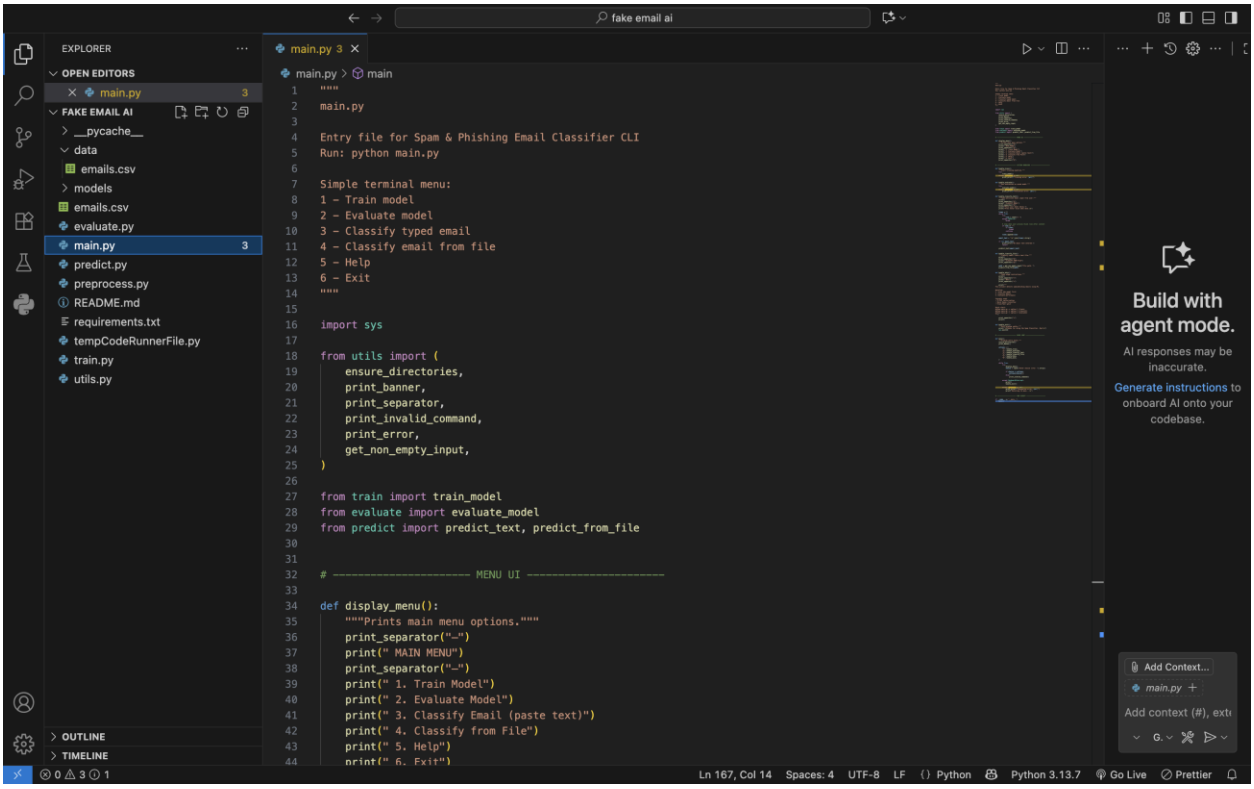
SOFTWARE ARCHITECTURAL DIAGRAM

User Input
↓
CLI Interface (main.py)
↓
Preprocessing Module
↓
TF-IDF Vectorizer
↓
Naive Bayes Model
↓
Prediction Output

CODING

The system is implemented using modular Python files:

- main.py – CLI controller
- train.py – Model training
- predict.py – Email prediction
- evaluate.py – Model evaluation
- preprocess.py – Text processing
- utils.py – Helper functions


fake email ai
EXPLORER
OPEN EDITORS
main.py 3
FAKE EMAIL AI
__pycache__
data
emails.csv
models
emails.csv
evaluate.py
main.py 3
predict.py
preprocess.py
README.md
requirements.txt
tempCodeRunnerFile.py
train.py
utils.py
OUTLINE
TIMELINE
main.py 3
main.py > main
"""
main.py
Entry file for Spam & Phishing Email Classifier CLI
Run: python main.py
Simple terminal menu:
1 - Train model
2 - Evaluate model
3 - Classify typed email
4 - Classify email from file
5 - Help
6 - Exit
"""
import sys
from utils import (
ensure_directories,
print_banner,
print_separator,
print_invalid_command,
print_error,
get_non_empty_input,
)
from train import train_model
from evaluate import evaluate_model
from predict import predict_text, predict_from_file
# -------------------- MENU UI --------------------
def display_menu():
"""Prints main menu options."""
print_separator("-")
print(" MAIN MENU")
print_separator("-")
print(" 1. Train Model")
print(" 2. Evaluate Model")
print(" 3. Classify Email (paste text)")
print(" 4. Classify from File")
print(" 5. Help")
print(" 6. Exit")
Build with agent mode.
AI responses may be inaccurate.
Generate instructions to onboard AI onto your codebase.
Add Context...
main.py
Add context (#), ext
Ln 167, Col 14
Spaces: 4
UTF-8
LF
Python
Python 3.13.7
Go Live
Prettier

OUTPUT

The following screenshots show execution results.

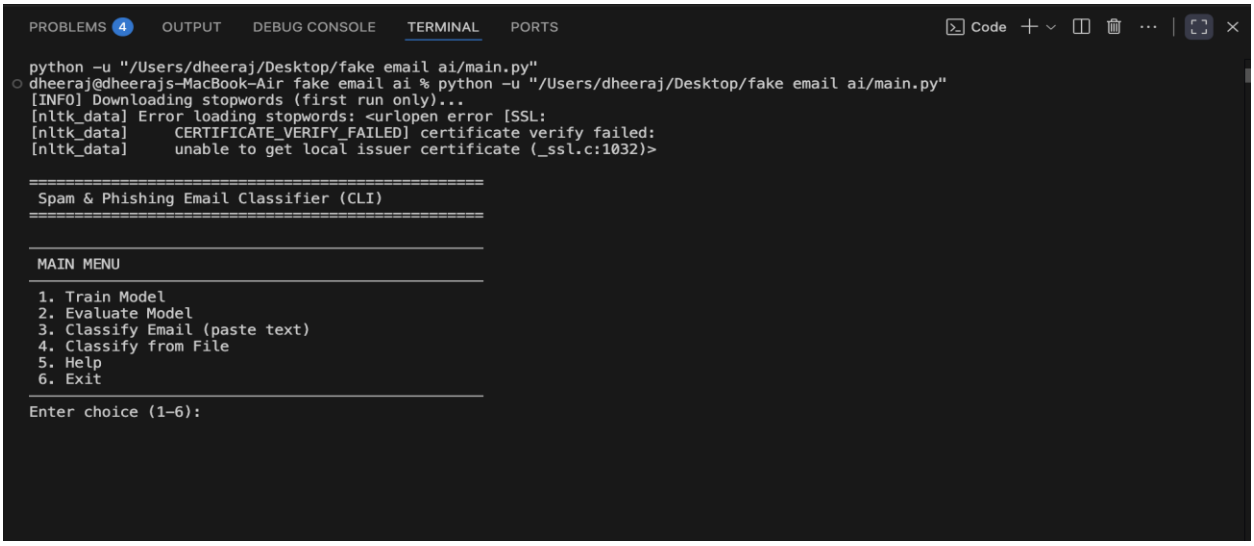


Figure: CLI Menu Interface

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

python -u "/Users/dheeraj/Desktop/fake email ai/main.py"
dheeraj@dheerajs-MacBook-Air fake email ai % python -u "/Users/dheeraj/Desktop/fake email ai/main.py"
[INFO] Downloading stopwords (first run only)...
[nltk_data] Error loading stopwords: <urlopen error [SSL:
[nltk_data]     CERTIFICATE_VERIFY_FAILED] certificate verify failed:
[nltk_data]     unable to get local issuer certificate (_ssl.c:1032)>

================================================
 Spam & Phishing Email Classifier (CLI)
================================================

------------------------------------------------
 MAIN MENU
------------------------------------------------
 1. Train Model
 2. Evaluate Model
 3. Classify Email (paste text)
 4. Classify from File
 5. Help
 6. Exit
------------------------------------------------
Enter choice (1-6): 3

------------------------------------------------
 CLASSIFY EMAIL
------------------------------------------------
Paste your email below.
Press Enter twice when done.

> Dear Team,I hope you are doing well.I am writing to share the latest progress on our CLI-based project. The core features have been
 implemented successfully, including user authentication, task management, and report generation.We are currently working on improvin
g error handling and adding additional test cases. Please review the updates and share your feedback.Best regards,Dheeraj
>

------------------------------------------------
✅ CLASSIFICATION RESULT
------------------------------------------------
Source     : Direct Input
Verdict    : HAM (Legitimate)
Confidence : 66.5% [                    ]
------------------------------------------------
Looks safe based on model prediction.

------------------------------------------------
 MAIN MENU
------------------------------------------------
 1. Train Model
 2. Evaluate Model
 3. Classify Email (paste text)
 4. Classify from File
 5. Help
 6. Exit
------------------------------------------------
Enter choice (1-6):
```

Figure: Spam Prediction Result

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- NLP preprocessing
- TF-IDF feature extraction
- Naive Bayes classification
- Model saving using Pickle
- Interactive CLI system

SIGNIFICANT PROJECT OUTCOMES

- Accurate spam email classification
- Automated detection system
- Practical understanding of ML lifecycle
- Real-world AI implementation

TESTING AND REFINEMENT

Testing methods:

- Train-test data split
- Performance metric evaluation
- Manual testing using sample emails

Refinements included improved preprocessing and error handling.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

- Email service providers
- Cybersecurity systems
- Corporate email filtering
- Fraud detection platforms

CONTRIBUTION / FINDINGS OF THE PROJECT

- Machine Learning significantly improves spam detection.
- NLP techniques enhance text classification accuracy.
- CLI applications can effectively deploy AI models.

LIMITATION / CONSTRAINTS OF THE SYSTEM

- Small dataset size
- Limited phishing pattern detection
- No graphical interface
- Requires retraining for new data

CONCLUSIONS

The project successfully demonstrates the use of Machine Learning and NLP techniques to classify spam emails automatically. The developed system achieves high accuracy and provides a practical implementation of AI concepts learned in the course.

FUTURE ENHANCEMENTS

- Web-based interface using Streamlit
- Deep learning models (LSTM/BERT)
- Larger datasets
- Real-time email integration
- Explainable AI visualization

REFERENCES

1. Scikit-learn Documentation — https://scikit-learn.org
2. Python Documentation — https://www.python.org
3. NLTK Documentation — https://www.nltk.org
4. IEEE Research Papers on Spam Detection
5. Hands-On Machine Learning by Aurélien Géron